

Ch 1

* Fig 1.1 - 1.5

* Decomposition of a compiler log phase

⇒ code optimization $O(n^2)$ $O(n^2)$ $O(n^2)$
(1.2.5)

ch2

* Syntax definition (production rules)
 ↳ if ~~for~~ an expression and

→ context free grammar 46 point

* Parse Tree ***

* Exercise 2.2.1, 2.2.2, 2.2.7

* 2.2 এর Exercise (51 page)

Ch 3

* Lexical Analysis vs Parsing

* 3.2 Input Buffering

* 3.6 - Finite Automata ***

→ NFA, DFA → 5% stop আছে পারে

* 3.6.1 - 3.6.3

* Fig: 3.24 - 3.25, - 3.26

 CH_4

✱

* 4.1.2, 4.2.3, 4.2.5

* 4.4 Top down Parsing

4.4.2 - First follow ***

* ਕਿਸੇ $LL(1)$ construct ਕਰੋ

ch 5

* 5.1, 5.1.1 (Rules), 5.1.2, 5.2, 5.2.1, 5.2.3, 5.2.4, 5.3.1, 5.4.3, 5.4.4

* Fig: (5.1.2) (a-g + e)

Ch6

* 6.11

* 6.1.1 (DAG) RegEx থেকে বিস্মৃতি DAG সৃষ্টি

* 6.1.2

* 6.2 ^{three} (preaddress code construction - fastest शीत) ***

* 6.2.1 unification

* 6.5.5 (Verification algorithm)

* 6.6 (Controlflow for statement for?)

* 6.9

* 6.22 (Exercise) (6.2.22 2008 पाठ)

Mid Term = Ch 2,5

[Figures in the right margin indicate full marks. Split answering of any question is not recommended]

Answer any 5 of the following questions.

1. (a) What does compiler mean? How do you distinguish between a compiler and an interpreter? 2+3=5
 (b) Illustrate the typical decomposition of a compiler into phases. Identify the *front end* and the *back end* parts in the structure of the compiler. 2+2=4
 (c) Suppose a source program contains the assignment statement: $position = initial + rate * 60$, show the representation of the assignment statement using lexical analyzer. 5

 2. (a) What is syntax-directed translation? Make the difference between tokens and terminals. 2+2=4
 (b) Define production. Explain the production: $stmt \rightarrow if(expr) stmt else stmt$. 2+2=4
 (c) Define context-free grammar including four components. Consider the context-free grammar $S \rightarrow SS + | SS * | a$ 3+3=6
 (i) Show how the string: $a a + a *$ can be generated by this grammar.
 (ii) Construct a parse tree for this string.

 3. (a) What is the role of lexical analyzer? Represent the interaction between lexical analyzer and parser. 2+2=4
 (b) Define regular expression with example. How to build a regular expression using basis and induction? Describe the language denoted by the regular expression: $a(a|b)^*a$. 4+2=6
 (c) Give a formal definition of Nondeterministic Finite Automata (NFA). Draw a transition graph for an NFA recognizing the language of regular expression $(a|b)^*ab b$ with the following transition table. 2+2=4
- | STATE | a | b | ϵ |
|-------|---------------|---------------|---------------|
| 0 | {0,1} | {0} | $\varnothing$ |
| 1 | $\varnothing$ | {2} | $\varnothing$ |
| 2 | $\varnothing$ | {3} | $\varnothing$ |
| 3 | $\varnothing$ | $\varnothing$ | $\varnothing$ |
4. (a) Illustrate the position of the parser in compiler model. What is ambiguous grammar? 2+2=4
 (b) Explain FIRST and FOLLOW functions for constructing of both top-down and bottom-up parsers. 2+2=4
 (c) Define predictive parser. What is LL (1) grammar? How to construct a predictive parsing table? 3+3=6

 5. (a) Distinguish between S-attributed definition and L-attributed definition. 4
 (b) What is dependency graph? Show a dependency graph from the production $E \rightarrow E_1 + T$ and semantic rule $E.val \rightarrow E_1.val + T.val$. 2+2=4
 (c) Explain the steps in the construction of the syntax tree for $a - 4 + c$, where general and simple expression including production and semantic rules can be considered. 6

 6. (a) What is intermediate-code generation? Show the logical structure of a compiler front end. 2+2=4
 (b) Define directed acyclic graph (DAG). Construct the DAG for the expression $a + a * (b - c) + (b - c) * d$. 2+2=4
 (c) What is three-address code? Build a three-address code from the expression $a + a * (b - c) + (b - c) * d$. 2+2=4
 (d) Design a control-flow translation from the statement: $if(x < 100 || x > 200 \ \&\& \ x \neq y) \ x = 0$. 2